

APIs as the Front Door to AI Models

Building Resilient Model Abstraction Layers - One API contract. Any model. Zero client changes.

ADO TRAKIC

Head of Platform and Innovation | Iron
Sheepdog

www.linkedin.com/in/ado-trakic



MADHURI MARGAM

Director, Software Engineering | Capital One

[www.linkedin.com/in/
madhuri-margam-4370a8150/](https://www.linkedin.com/in/madhuri-margam-4370a8150/)



API Days

New York City, May 2026

The AI Model Landscape is Accelerating

RELEASED	VENDOR	MODEL	SIGNIFICANCE
Apr 2025	OpenAI	o3 + o4 mini	Next reasoning generation; strong math, science & coding
Apr 2025	Meta	Llama 4	New open-weight generation; 10M-token context; multimodal
Apr 2025	Alibaba	Qwen 3	New open-source generation; competitive across multiple scales
May 2025	Anthropic	Claude 4 Opus	New Claude generation; hybrid reasoning; 7hr autonomous operation
Jul 2025	xAI	Grok 4	SOTA on ARC-AGI v2 & Humanity's Last Exam at launch
Aug 2025	OpenAI	GPT-5	New flagship generation; adaptive reasoning; reduced hallucinations
Nov 2025	Google	Gemini 3.0	Launched Gemini 3 generation; #1 on major benchmarks at release
Feb 2026	Anthropic	Claude 4.6 (Opus + Sonnet)	Agent Teams; 1M-token context; near-Opus quality at lower cost
Feb 2026	Google	Gemini 3.1 Pro	2x reasoning improvement; ARC-AGI-2 77.1%; 2M-token context
Apr 2026	OpenAI	GPT-5.4 family	Latest OpenAI generation; flagship + mini + nano tiers

10+ Models

Major foundation model releases per year across providers. Your production code is coupled to *one* of them.

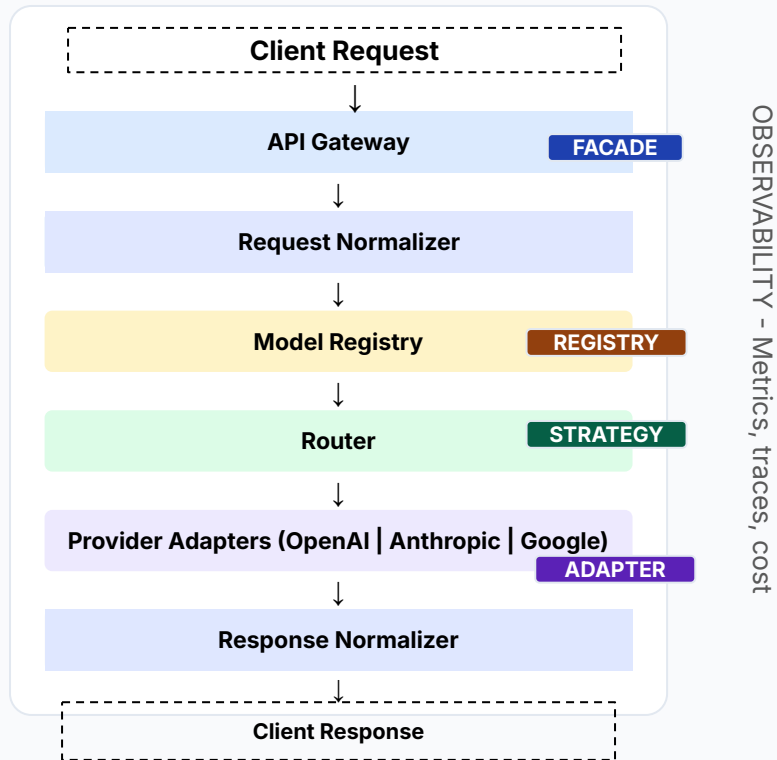
The Problem

Tight Coupling

Constant breaking changes caused by model-specific codebases.

Warning Hard-coded providers make switching or upgrading models a manual, error-prone effort.

Architecture & Design Patterns



Why These Patterns

Facade Pattern

One endpoint for all providers.

Registry Pattern

Model mappings live in config, not code.

Strategy Pattern

Swap routing logic without touching adapters.

Adapter Pattern

One interface per provider.

Unified API & Model Naming

API Contract

OpenAI Chat Completions format adopted as the de facto standard for the unified interface.

```
// REQUEST

POST /v1/chat/completions

{
  "model": "default-reasoning",
  "messages": [
    {
      "content": "Explain
Quantum....",
    }
  ],
  "temperature": 0.7,
  "max_tokens": 500
}

// RESPONSE

{
  "id": "chat-abc123",
  "model": "default-reasoning",
  "choices": [
    {
      "message": {
        "content": "Quantum..."
      }
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 245,
    "total_tokens": 257
  }
}
```

Logical Model Naming

Logical Model	Current Mapping	Use Case
default	Claude Sonnet 4.6	General purpose
default-reasoning	Claude Opus 4.6	Complex tasks
default-fast	Claude Haiku 4.5	High volume
default-vision	GPT-4o	Image analysis
default-long	Gemini 1.5 Pro	100K+ tokens

Key Design Decisions

- Standard message format
- Unified streaming interface
- Consistent error handling
- Provider-agnostic response

Model Abstraction Layer

Registry Pattern — Config as Data

models:

default:

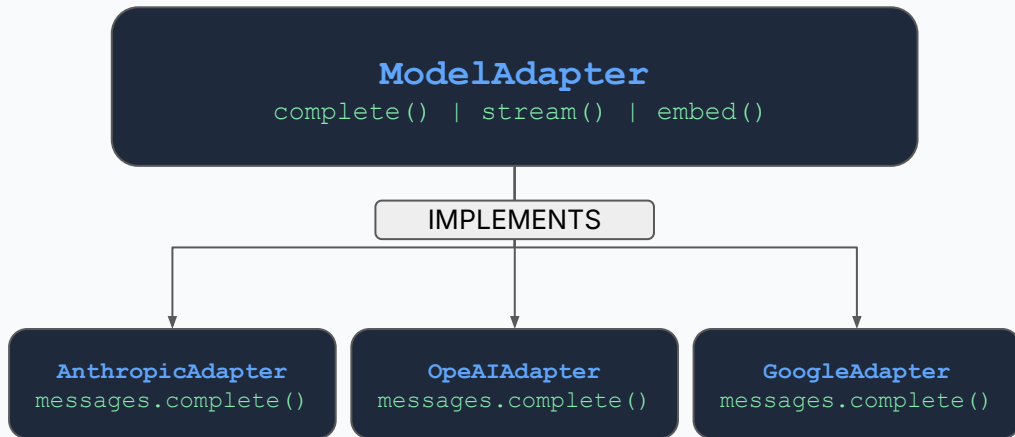
```
provider:    anthropic
model_id:    claude-sonnet-4-6-20250514
max_tokens:  8192
capabilities: [chat, streaming]
```

default-reasoning:

```
provider:    anthropic
model_id:    claude-opus-4-6-20250514
max_tokens:  16384
capabilities: [chat, streaming, vision]
```

default-fast:

```
provider:    anthropic
model_id:    claude-haiku-4-5-20251001
max_tokens:  4096
capabilities: [chat, streaming]
```



Adding a new provider? One class, three methods.
No changes to routing, registry, or the API layer.

Smart Routing

ROUTING CONFIGURATION

```
# routing_config.yaml
```

```
routes:
  default-reasoning:
    fallback:
      - provider: openai
        model: gpt-4o
      - provider: google
        model: gemini-2.5-pro

  health:
    failure_threshold: 5
    recovery_timeout: 30s
    probe_interval: 10s
```

Primary comes from registry. Router defines fallbacks and health thresholds.

Client Request: "model": "default-reasoning"

Primary: Anthropic
claude-opus-4-6

× 529 overloaded

Fallback 1: OpenAI
gpt-4o

× 503 unavailable

Fallback 2: Google
gemini-2.5-pro

✓ 200 OK

Two providers failed, but the client never knew. Same logical name, same response format, **zero disruption**.

Response Normalization

Anthropic

```
CONTENT :Content[0].text  
ROLE  : "assistant"  
Tokens:input/output_tokens
```

OpenAI

```
Content:choices[0].message.content  
Role  : "assistant"  
Tokens:prompt/completion_tokens
```

Google Gemini

```
Content: Candidates[0].content.  
          parts[0].text  
Role: "model"  
Tokens:prompt/candidatesTokenCount
```

Adapter Normalizer

Maps paths, roles, and token counts to a standard schema.

Unified Response

```
CONTENT :  
choices[0].message.content  
  
ROLE: "assistant"  
  
TOKENS: prompt_tokens /  
completion_tokens /  
total_tokens
```

Three formats in, one format out. Content paths, role naming, token fields — the adapter translates all of it.

Streaming: Normalizing SSE streams is significantly harder than single JSON responses.

Error Normalization

Three Providers, Three Formats — One Contract

Provider	Original Error		Unified Output
OpenAI	429 rate_limit	→	429 rate_limited
Anthropic	529 overloaded	→	503 model_unavailable
Google	429 RESOURCE_EXHAUSTED	→	429 rate_limited
OpenAI	404 model_not_found	→	400 invalid_model

Why this matters: Multiple formats, different conventions, clients see one standard response.

```
// All providers normalize to this
{
  "error": {
    "code":
      "model_unavailable",
    "Message": "provider is temp
overloaded",
    "status": 503,
    "provider": "providerName",
    "retry_after": 30,
    "request_id": "req-xyz-789"
  }
}
```


Cost Optimization

Per-Token Pricing by Model

Model	Input / 1M	Output / 1M
Claude Opus 4.6	\$5.00	\$25.00
Claude Sonnet 4.6	\$3.00	\$15.00
GPT-4o	\$2.50	\$10.00
Claude Haiku 4.5	\$1.00	\$5.00
Gemini 2.0 Flash	\$0.10	\$0.40

Illustrative Example

Hypothetical: Summarization service — 10M output tokens /day
80% simple (short docs, emails) | 20% complex (legal, financial)

ALL → SONNET
\$4,500
per month

SMART ROUTING
\$2,100
per month

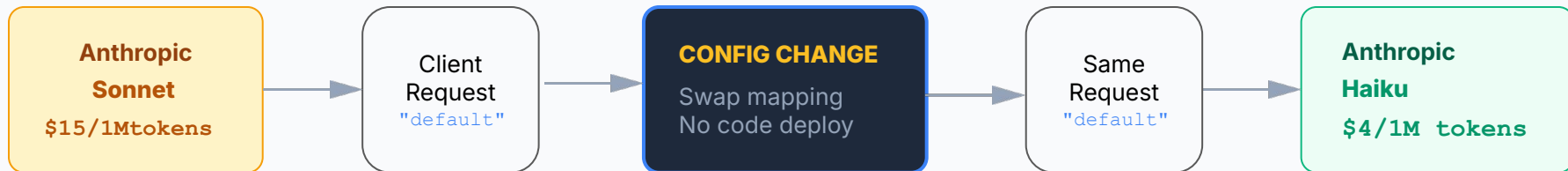
SAVINGS
\$2,400
53%

Smart routing breakdown

8M tokens/day → Haiku × \$5.00 /1M	\$1200 /mo
2M tokens/day → Sonnet × \$15.00 /1M	\$900 /mo

Immute — Live Demo

The client-facing contract never mutates. One API contract. Any model. Zero client code changes.



demo — live terminal

1	2	3	4
Send request <code>model: "default"</code> → routed to Sonnet show tokens + cost	Edit registry YAML <code>swap to Haiku</code> one line change no restart needed	Same request again <code>model: "default"</code> → routed to Haiku show tokens + cost	Compare results <code>same task completed</code> actual cost difference from real API responses

Try Immute → <https://github.com/vw4motion/Immute-api-days-nyc-2026/blob/main/immune-poc.html>

Before You Build This

01 Model Are not Interchangeable

Same prompt, different model, different output.

→ *Validate quality before and after every swap.*

02 Prompts have Model Affinity

Tuned prompts behave differently across models.

→ *Test across target models before routing.*

03 Unified = Lowest Common Denominator

API can only expose features all providers share.

→ *Scope to shared features. Use passthroughs.*

04 Streaming is the hard part

Every provider streams differently.

--> *Start synchronous. Add adapters later.*

Key Questions Before You Build

- How many providers do you actually need today?
- Can use cases tolerate output variance?
- Which provider-specific features are critical?
- Is synchronous-only acceptable for v1?
- Does an existing tool cover 80% of needs?

The patterns matter more than the implementation.

Whether you build or buy, the architecture decisions are the same.

Key Takeaways

Takeaway 1

Abstract early, not later

Logical model names from day one. Retrofitting an abstraction layer onto hardcoded provider calls is painful.

Takeaway 2

Route by task, not by habit

Classify request complexity and route accordingly. Not every request needs your most expensive model.

Takeaway 3

Config, not code

Model swaps, fallbacks, and routing changes should be a config change. Invest in hot-reload early.

Takeaway 4

Evaluate before you build

Multiple Solutions exist in the market. The patterns are the same. Pick the right starting point.

Q&A

Thank you! Questions?

Feedback

